

ChaosProbe thesis review and experiment strategy

Executive summary

The updated repository is **much closer to a defensible MSc thesis** than it was before. The strongest improvements are exactly the ones a hostile examiner would have demanded: the repo now narrows the core matrix to **three strategies** (`default` , `colocate` , `spread`) across **two fault classes** (`pod-delete` churn and `node-memory-hog` contention), adds a **scope-of-claims** page, records **batch/day IDs**, persists **scenario SHA-256 hashes**, records **kube-proxy mode and conntrack settings**, and adds **EndpointSlice snapshots** as first-party API evidence for the churn/reconvergence story. Those are not cosmetic tweaks; they materially strengthen provenance, reproducibility, and claim scoping. ¹

My bottom line is blunt: **yes, this can be a worthy and defensible MSc systems thesis**, but only if you frame it as a **bounded cross-layer measurement study** of how **fault class** and **measurement layer** interact with **placement**, rather than as a universal placement-ranking paper, a scheduler paper, or a proof that some strategy is “best.” The right thesis is not “spread beats colocate” or “placement predicts resilience.” The right thesis is: **placement effects under chaos are fault-class-specific and measurement-layer-specific; in at least some regimes, mechanism metrics move while user-visible outcomes do not, and coarse aggregate scores can therefore mislead**. That framing aligns with your updated docs, avoids overclaiming, and gives you an argument that is both interesting and exam-proof. ²

The **single best new experiment** for a fast but strong thesis is the one your updated docs already point toward: `node-memory-hog` **targeted at the node hosting** `productcatalogservice` **with** `TARGET_NODES=auto` . Kubernetes enforces **CPU** limits by throttling, but **memory** limits are enforced reactively and node memory pressure can trigger **evictions** and OOM-related disruption; the Litmus node-memory-hog experiment is explicitly designed to test resilience under node memory exhaustion. That makes node-memory-hog the cleanest placement-sensitive contention fault in your current stack. By contrast, `pod-cpu-hog` **is a bad headline experiment** for this thesis, and your docs are right to demote it. ³

There are still **three serious blockers**. First, the committed thesis results folder I could inspect contains **charts only**, while your own docs say thesis runs should be reproduced from `results/<run>/summary.json` and gated with `doctor --strict` ; that means the public repo still lacks the raw thesis evidence chain. Second, your analysis scripts still define churn in a way that excludes `pod-cpu-hog` but can **misclassify** `node-cpu-hog` **as churn** because the helper checks for `cpuhog` without the hyphen while the scenario name is `node-cpu-hog` . Third, there is still **doc drift**: the Online Boutique scenario README calls pod-delete a scheduling-contention experiment on `checkoutservice` , जबकि the actual `pod-delete.yaml` targets `productcatalogservice` and your thesis docs now treat it as a **churn / reconvergence** fault. Fix those three things before you trust any thesis claim. ⁴

Judgment	Verdict
Thesis-worthiness	Yes , if framed as a bounded empirical systems study, not a universal placement-ranking paper
Strongest title	When Placement Matters Under Chaos: A Cross-Layer Measurement Study of Fault Class and Placement in Kubernetes Microservices
Best single new core experiment	<code>node-memory-hog</code>
Best minimal core matrix	<code>default</code> , <code>colocate</code> , <code>spread</code> × <code>pod-delete</code> , <code>node-memory-hog</code>
Weakest headline experiment	<code>pod-cpu-hog</code>
Biggest remaining risks	Missing raw archives, fault-class misclassification bug, doc drift / claim drift

Repo pull and audit

I reviewed the current public GitHub state of `yhu02/uva-msc-thesis` at **HEAD** `f5bb962`, with the recent visible change set concentrated on **June 6–7, 2026**. Since I do not have your earlier local clone in this session, I cannot prove a literal “since your last pull” diff; the safest interpretation is the **recent GitHub delta** visible in the repo history. That history shows a deliberate push toward thesis hardening: a run archiver and artifact manifest, a tighter core matrix, a scope-of-claims page, EndpointSlice snapshots, batch/day IDs, scenario hashes, and kube-proxy/contrack provenance capture. ⁵

The top-level repo currently contains the `chaosprobe/` tool, GitHub workflow config, an advisory report PDF, the presentation deck, a presentation script, and a references file. The root listing does **not** show a thesis manuscript source tree. Inside `chaosprobe/`, the visible structure includes docs, examples, results, scenarios, scripts, and a large tests tree. That means the repo already looks like a serious artifact repo, but it still does **not** look like a finished thesis repo because the manuscript and reviewer-grade raw data package are not present in the visible top level. ⁶

The updated commit stream is directionally excellent. The most important recent commits, from a thesis-defense perspective, are summarized below. The phrasing in the “Why it matters” column is my synthesis, but the commit identities and commit intentions come directly from the repo history and inspected commit pages. ⁷

Commit	What changed	Why it matters for the thesis
<code>7dd1061</code>	Added run archiver + artifact manifest; restored doc drift-guard	This is the right answer to provenance objections, but you still need to use it and publish the artifacts

Commit	What changed	Why it matters for the thesis
<code>ce7518d</code>	Thesis docs now define the core matrix as 3 strategies × 2 fault classes	Strong move: narrower matrix means more power and less cherry-picked breadth
<code>fc6fb86</code>	Presentation claims were scoped down	Good defense hygiene; slides are often where overclaims survive
<code>381b968</code>	Added an explicit scope-of-claims page	Very good; this should govern manuscript wording
<code>ce0fd25</code>	Added EndpointSlice endpoint-count snapshots around fault windows	Excellent mechanism evidence for churn/reconvergence claims
<code>85b92f9</code>	Added batch/day ID as run metadata	Necessary if you want to block by day and discuss repeatability honestly
<code>e0debff</code>	Added scenario SHA-256 hashes and doctor gating	Critical for “exact scenario used” provenance
<code>2aed048</code>	Added kube-proxy mode + conntrack settings to run metadata	Critical because your mechanism claims are network-stack contingent
<code>8a4f65a</code>	Added reproducible H1/H3 scripts and hypotheses doc	Good, but claims reproduced from scripts are only as strong as the committed raw archives behind them

The most important **confirmed file-level delta** I inspected is commit `ce0fd25`. It changes `chaosprobe/TECHNICAL.md`, `chaosprobe/chaosprobe/metrics/collector.py`, `chaosprobe/chaosprobe/metrics/endpointslices.py`, `chaosprobe/chaosprobe/orchestrator/strategy_runner.py`, `chaosprobe/pyproject.toml`, `chaosprobe/tests/test_collector.py`, and `chaosprobe/tests/test_endpointslices.py`. That is exactly the right file set for injecting API-level churn observability into the evidence chain. ⁸

The repo’s **largest remaining evidence-chain problem** is still raw artifact absence. Your reproduction doc says the defense results should be regenerated from `summary.json` files, gated by `doctor --strict`, and statistically processed from those summaries. But the visible committed thesis results folder `chaosprobe/results/20260606-092037/` contains only a `charts/` directory, and that directory contains PNGs, a CSV, and an HTML report, not the raw run summaries or per-iteration archives. The examples directory does ship a toy `example-summary.json`, but that is clearly a demo fixture, not thesis evidence. A hostile examiner can therefore still say: “show me the raw run.” Right now, on the public repo, you cannot. ⁹

The repo also still contains a **real analysis bug** that can poison thesis scripts. In all three visible analysis scripts, churn is defined through a helper that returns true when `"cpuhog" not in fault_name and fault_name != "pod-cpu-hog"`. That excludes `pod-cpu-hog`, but it does **not** exclude `node-cpu-hog`, because the actual scenario name contains a hyphen: `node-cpu-hog`. If a directory scan sees node-cpu-hog summaries, they can be silently included in the churn set. That is exactly the kind of

“one ugly helper function destroyed your conclusion” bug a hostile examiner loves. Fix it centrally and add tests immediately. ¹⁰

The minimum concrete code fix is small and should be done before any new thesis plots are trusted:

```
# chaosprobe/scripts/fault_taxonomy.py

def normalize_fault_name(name: str) -> str:
    return name.lower().replace("_", "-").strip()

def fault_class(name: str) -> str:
    n = normalize_fault_name(name)
    if n == "pod-delete":
        return "churn"
    if "cpu-hog" in n:
        return "cpu-contention"
    if "memory-hog" in n:
        return "memory-contention"
    if "network-latency" in n or "network-loss" in n:
        return "network"
    if "io-stress" in n or "disk" in n:
        return "io"
    return "other"

def is_churn(name: str) -> bool:
    return fault_class(name) == "churn"
```

Then import that helper everywhere and add a parametrized test over `pod-delete`, `pod-cpu-hog`, `node-cpu-hog`, `node-memory-hog`, `pod-network-loss`, and `pod-io-stress`. That recommendation follows directly from the current helper logic and the now-present `node-cpu-hog` scenario. ¹⁰

There is also still **doc drift** that weakens the repo’s credibility. The scenario README still describes `pod-delete` as “Scheduling Contention” targeting `checkoutservice`, but the actual `pod-delete.yaml` targets `productcatalogservice`, and the newer thesis docs explicitly treat `pod-delete` as the **churn** fault class. That inconsistency is easy to fix and should be fixed now. Do not make an examiner figure out which document is current. ¹¹

One positive sign is that the tests tree is already substantial: there are visible tests for archiving, doctor, EndpointSlice collection, node-target resolution, reproducibility, and many core CLI/data-pipeline functions. But what I do **not** see in the visible tests index is dedicated test coverage for the **thesis analysis scripts** themselves. That is a gap, because your dissertation’s load-bearing claims currently live in script logic, not just in the core CLI. The closer the logic is to the thesis claim, the less acceptable it is for it to be untested. ¹²

If you want to re-pull and verify the exact Git state locally, these are the commands I would use:

```

git clone https://github.com/yhu02/uva-msc-thesis.git
cd uva-msc-thesis

git rev-parse HEAD
git rev-parse --short=12 HEAD
git log --oneline --decorate -n 15

# Verify the recent thesis-hardening commits
git show --stat f5bb962
git show --stat 7dd1061
git show --stat ce7518d
git show --stat 381b968
git show --stat ce0fd25
git show --stat 85b92f9
git show --stat e0debff
git show --stat 2aed048

# See current name-status changes by date range if you already had a prior clone
git log --since="2026-06-06" --name-status -- chaosprobe

```

Thesis worthiness and framing

As a thesis, ChaosProbe is strongest when it is presented as **an empirical systems artifact plus a bounded measurement study**, not as a new scheduler and not as a claim that one placement policy “wins” in general. That distinction matters because the literature you need to align with splits cleanly into three camps: **cluster scheduling / placement** papers that optimize for QoS or utilization, **chaos engineering** work that automates fault injection and resilience testing, and **microservice resilience profiling** work that tries to connect mechanism-level degradation with user-aware impact. ChaosProbe sits at the intersection, but its distinctive value is not beating Borg, Quasar, Sparrow, or Medea on scheduling quality. Its value is showing **where placement-sensitive effects appear under chaos, and where they do not.** ¹³

The strongest single research question is:

In a Kubernetes microservice application, under which chaos fault classes does pod placement measurably affect mechanism-level behavior, user-visible outcomes, and aggregate probe scores, and when do these measurement layers diverge?

That question is strong because it is falsifiable, it matches the updated repo direction, and it turns your likely best finding into a contribution rather than an embarrassment. If the aggregate score is noisy or misleading, that is not failure; it is a result about measurement. If mechanism metrics move but user outcomes do not, that is not failure; it is a result about observability and construct validity. ¹⁴

The strongest title is:

When Placement Matters Under Chaos: A Cross-Layer Measurement Study of Fault Class and Placement in Kubernetes Microservices

A slightly narrower but even safer title, if you end up with only the quick core experiments, is:

Placement Under Churn and Contention: A Cross-Layer Chaos Study of Kubernetes Microservices with ChaosProbe

Both titles are better than “resilience score” titles because they foreground **measurement**, **fault class**, and **scope**, instead of implying universal conclusions. That is exactly the spirit of your new scope-of-claims page.

15

The best contribution statement is this:

ChaosProbe is an open-source framework for running placement-mutated chaos experiments on Kubernetes microservices and collecting cross-layer evidence from LitmusChaos, Kubernetes, Prometheus, and load generation.

Using ChaosProbe on Google Online Boutique, this thesis shows that **placement effects are fault-class-specific and measurement-layer-specific**: in a single-replica churn regime, placement can reproducibly alter reconvergence mechanisms without reliably altering user-visible outcomes, while coarse aggregate probe scores can be too noisy to rank strategies.

The thesis also introduces and evaluates a **node-memory-hog** experiment design that better targets placement-sensitive contention by operating at the node-pressure / eviction layer rather than at container-local CPU throttling. 16

I would separate your thesis contributions as follows:

Contribution type	What you can honestly claim
Novel research contribution	A cross-layer fault-class × placement measurement study showing that placement effects do not transfer uniformly across aggregate scores, mechanism metrics, and user-visible outcomes
Engineering contribution	The ChaosProbe artifact: placement mutator, Litmus runner, probers, aggregation, doctor/stats/report/export commands, Neo4j storage, tests, docs
Replication / validation contribution	A reproducibility protocol with batch/day IDs, scenario hashes, kube-proxy/contrack provenance, EndpointSlice snapshots, archiving, and strict gating
Pilot contribution	<code>pod-cpu-hog</code> , <code>node-cpu-hog</code> , wide strategy matrix, and any exploratory appendix-only runs

One more blunt point: **Neo4j is not a research contribution unless the thesis actually uses it analytically**. If Neo4j is just the storage substrate that makes the tool nicer, call it an engineering contribution. Do not put it in the abstract unless you use it to answer a research question. The same goes for AI-driven anomaly classification and remediation language in the root README: if the thesis results do not depend on that pipeline, keep it out of the contribution statement. 17

Literature positioning

The cleanest way to position the thesis is to say that it builds on **scheduler/placement** literature, **Kubernetes traffic-engineering primitives**, **chaos-engineering methodology**, and **microservice resilience profiling**, but asks a different question from all four.

Kubernetes' own placement primitive, **Pod Topology Spread Constraints**, is motivated by spreading Pods across failure domains to improve availability and resource use. The docs explicitly talk about replicated workloads and avoiding colocating too many Pods on one node. That matters because your current core pod-delete case uses **single replicas**, so you should not imply that you are directly validating or invalidating the main availability promise of topology spread. You are testing something narrower: whether **placement-induced locality** changes the collateral effects and reconvergence behavior of faults in a microservice graph. ¹⁸

Kubernetes networking now centers on **EndpointSlices**, which scale backend endpoint tracking and expose endpoint conditions such as **serving** and **terminating**. kube-proxy consumes those updates and exports a **network programming duration** metric defined as the time from a service/pod change until the appropriate kube-proxy rules are synced; the code comments also note that the metric can be distorted by control-plane / node clock drift. This is exactly why your new EndpointSlice snapshots and kube-proxy-mode/conntrack provenance capture are valuable: they anchor the churn story in first-party Kubernetes control-plane evidence rather than in hand-wavy interpretation. ¹⁹

At the Linux/Kubernetes resource-control layer, the asymmetry between **CPU** and **memory** is central to your experiment design. Kubernetes documents that **CPU limits are enforced by throttling**, while **memory limits are enforced via OOM-related killing under pressure**; Kubernetes also documents node-pressure eviction thresholds, with **hard memory eviction** causing immediate pod termination once thresholds are met. That technical asymmetry strongly supports your move away from pod-level CPU hog as a core contention fault and toward node-level memory pressure as the cleaner placement-sensitive contention mechanism. ²⁰

On the chaos-engineering side, the **Principles of Chaos Engineering** emphasize forming a hypothesis around steady state and then running controlled experiments against turbulent conditions. LitmusChaos is a Kubernetes-native chaos platform with official pod-delete, node-memory-hog, pod-network-latency, pod-network-loss, and pod-io-stress experiments. Your thesis aligns well with that methodology, but the point of comparison is not "we built another chaos platform." The comparison point is that ChaosProbe contributes a **placement-aware, cross-layer measurement harness** on top of chaos execution. ²¹

MicroRes is the nearest academic relative to your framing. It injects failures, collects metrics, and builds a resilience index from the dissemination of degradation from system-performance metrics to user-aware metrics. That means your thesis should not pretend to be the first work connecting mechanism and user layers. The more accurate positioning is: ChaosProbe contributes a **placement-aware case study of when that dissemination linkage appears cleanly and when it breaks down**. That is a very good MSc-level angle. It is much safer than claiming to "refute" aggregate resilience metrics in general. ²²

ChaosEater is also relevant, but mainly as a foil. ChaosEater automates the chaos-engineering cycle with LLMs and evaluates whether it can complete reasonable CE cycles on Kubernetes systems at low time and monetary cost. Your work is not trying to automate the CE lifecycle; it is trying to make the **measurement**

question sharper. So the right contrast is: ChaosEater automates experiment planning and execution, while ChaosProbe focuses on **cross-layer evidence collection and placement-sensitive fault analysis**. ²³

The older and broader **placement / cluster management** literature gives you motivation, not a direct baseline. **Borg, Omega, and Kubernetes** captured the lessons of large-scale container management; **Sparrow** focused on low-latency distributed scheduling; **Quasar** made cluster management performance- and QoS-aware; **Medea** is especially close to your story because it explicitly studies placement constraints and shows that affinity/anti-affinity choices can materially change latency and throughput in shared clusters. That literature tells you that placement matters. Your thesis asks a narrower question: **does placement still matter under injected chaos, and at which measurement layer?** ²⁴

For benchmark positioning, **DeathStarBench** is still the strongest research benchmark lineage for microservice systems work, and many systems papers validate against it. Your current workload, **Google Online Boutique**, is a perfectly reasonable microservice artifact for an MSc thesis because it is a real multi-service e-commerce app with dependency structure and user-facing routes, but using a **single application** does constrain external validity. That means you should be explicit: this is a **deep case study** on Online Boutique, not a demonstration that the result generalizes across all microservices benchmarks. If you have extra time, even a tiny appendix cross-check on one DeathStarBench app would buy credibility, but it is not mandatory for a solid MSc thesis. ²⁵

Finally, the **tail-latency** literature helps you justify your metric choices. The “Tail at Scale” argument is that large fan-out services are dominated by slow tails rather than just means, and “Tales of the Tail” shows that poor tail latency emerges from multiple hardware, OS, and application factors. That supports your decision to prefer **route-level P95/P99/error-rate analysis** over coarse mean-only or pass/fail score summaries. In a microservice graph, that is exactly where placement-sensitive effects are most likely to show up first. ²⁶

Literature strand	Canonical anchors	What that literature usually optimizes / measures	Where ChaosProbe fits
Placement / scheduling	Borg, Sparrow, Quasar, Medea	QoS, utilization, scheduling latency, affinity / anti-affinity benefits	Uses placement as an experimental variable, not as an optimization target
Kubernetes availability / topology	TopologySpread, EndpointSlices, kube-proxy	Replica spread, endpoint propagation, traffic engineering	Uses these primitives/mechanisms as measurement context and evidence
Chaos engineering	Principles of Chaos, LitmusChaos, ChaosEater	Fault injection, CE workflows, automation	Adds placement-aware cross-layer evidence collection and analysis
Resilience profiling	MicroRes	System-to-user degradation dissemination and resilience indexing	Tests when this propagation story holds cleanly and when it does not

Literature strand	Canonical anchors	What that literature usually optimizes / measures	Where ChaosProbe fits
Microservice benchmarking / latency	DeathStarBench, Tail at Scale, Tales of the Tail	Representative apps, fan-out behavior, tail metrics	Justifies using route-level tail latency and scoped benchmark claims

Experiment portfolio and ranked recommendations

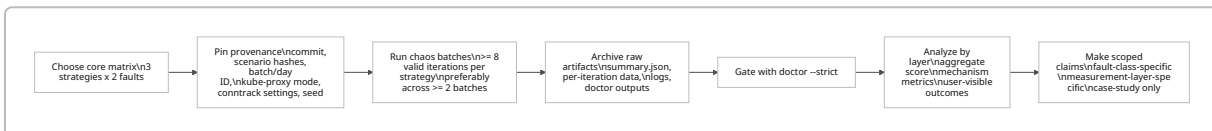
Your updated docs already made the right strategic move by shrinking the core matrix to `default`, `colocate`, and `spread`, and by demoting the wider eight-strategy matrix to appendix status. That is statistically and narratively correct. The more strategies you compare on a small testbed, the more likely you are to produce a muddy table that says little. Three strategically distinct placements are enough for a thesis: a control (`default`), a contention-maximizing extreme (`colocate`), and a locality / interference-minimizing extreme (`spread`). ²⁷

The same narrowing logic should guide fault choice. Not all chaos faults are equally informative for **placement-dependent** results. Your current fault candidates are not interchangeable. Some test the right layer; some do not. Some are likely to saturate the outcome and wash out placement differences; some are likely to produce an actual gradient that a thesis can measure. The ranking below is my synthesis from the repo and the underlying systems literature. ²⁸

Fault / design	Headline verdict	Novelty	Defensibility	Feasibility	Expected signal strength	Risk
<code>node-memory-hog</code> on <code>productcatalog</code> host node	Best core contention experiment	High	High	High	High	Medium
<code>pod-delete</code> on single- <code>replica</code> <code>productcatalogservice</code>	Best already-supported churn experiment	Medium	High	High	Medium at mechanism layer, low-medium at user layer	Low
Destination-scoped network latency / loss	Best ambitious extension if redesigned path-specifically	High	Medium	Medium	High if path-scoped, low if current pod-level only	High

Fault / design	Headline verdict	Novelty	Defensibility	Feasibility	Expected signal strength	Risk
<code>node-cpu-hog</code>	Useful appendix falsification / smoke test, not headline	Medium	Medium	High	Low-medium	Medium
Redis latency	Interesting only if placement mutator explicitly models the cart↔Redis pair	Medium	Medium	Medium	Medium	Medium-high
Disk I/O stress on <code>productcatalogservice</code>	Probably boring / weak for this workload	Low	Low	High	Low	High
<code>pod-cpu-hog</code>	Do not use as headline	Low	Low	High	Low	Low

Experiment workflow



Node-memory-hog should become the main contention experiment

Hypothesis. Under node-scoped memory pressure targeted at the node hosting `productcatalogservice`, `colocate` will produce a larger blast radius and worse user-visible degradation than `spread`, because dense placement concentrates more pods on the stressed node and memory pressure triggers evictions/OOM-related disruption at the node layer. This is the clearest top hypothesis in the updated repo. ²⁹

Independent variable. Placement strategy with three levels: `default`, `colocate`, `spread`. If you have extra time, a secondary IV is memory pressure intensity, e.g. 60%, 75%, and 85%, but that is an extension, not the core. The repo already documents a core scenario at **75%** memory consumption with `TARGET_NODES=auto` and one worker selected. ³⁰

Dependent variables. The primary DVs should be **user-visible** route outcomes on product-dependent paths: homepage and product-page availability/error rate, Locust-level failure counts, throughput, and route-level P95/P99 latency. Secondary DVs should be **mechanism metrics**: pod evictions, restart counts, OOM kill counts, node `MemoryPressure`, recovery time, and the number of affected pods on the stressed node. CPU throttling is not a primary DV here; if you see it, treat it as side context. ³¹

Controls. Keep the documented reference cluster fixed unless you explicitly study hardware sensitivity: one control plane plus four standardized 4 GiB workers, same CNI, same container runtime, same Kubernetes version, same load profile, same chaos duration, same target service, same seed, same scenario hash, same warmed deployment, and the same batch/day blocking logic. The reproduction doc already gives you a clean reference environment and warns against reintroducing heterogeneous-worker confounding. ³²

Expected mechanism. This experiment bites at exactly the layer where placement matters. Kubernetes documents that node-pressure eviction is how kubelet reclaims starved resources, with hard memory eviction immediately killing pods once thresholds are crossed; Kubernetes also documents that memory enforcement behaves differently from CPU limits, which are throttled. Your scenario comments correctly exploit that asymmetry. If the target node hosts almost all app pods under `colocate`, the blast radius should be large. If the target node hosts only a few relevant pods under `spread`, the damage should be more localized. ³³

Metrics to collect. At minimum archive: per-iteration route-level latency/error/throughput, node conditions, pod status snapshots, restart counts, OOM counts, evictions, Prometheus memory pressure / PSI if available, affected-pod counts per node, and the exact node chosen by `TARGET_NODES=auto`. Also save `doctor --strict` output and run metadata with scenario hashes, batch/day IDs, kube-proxy mode, and conntrack settings. If you only save final charts, this experiment does not count. ³⁴

Statistics. Use a **blocked design**. The cleanest version is a mixed-effects model or a blocked nonparametric design with **batch/day as a blocking factor** and strategy as a fixed effect. For pairwise comparisons, use **Holm-corrected Wilcoxon signed-rank** if you can pair iterations or batch medians cleanly; otherwise use Mann-Whitney with effect sizes and make the independence assumption explicit. Report effect sizes, bootstrap CIs, and not just p-values. For binary outcomes such as “route failed during chaos,” use logistic regression or exact/binomial confidence intervals. This is an inference from the structure of your runs and from the updated addition of batch/day IDs. ³⁵

Minimum repetitions / seeds. Absolute minimum: **8 valid iterations per strategy**, because your own docs set that threshold. Better: **10 valid iterations per strategy across at least two batch/day runs**. If you only have one batch, you may claim an internally consistent result, but not “reproducibility” in the broad sense. ³⁶

Threats to validity. The biggest here are environmental sensitivity of eviction thresholds, background cluster noise, exact free memory before each run, and the possibility that 75% memory pressure is either too weak or too strong on your cluster. You defend against that by archiving kubelet eviction config, worker memory headroom, batch/day IDs, and the full run metadata. ³⁷

Why this is academically interesting. This experiment gives you the best chance of a **clear placement-dependent user-level effect**, which is exactly what your current pod-delete result may fail to show. It also

cleanly demonstrates a systems point: **the mechanism layer you perturb matters**. That is good MSc material. ³⁸

Pod-delete is the best churn experiment, but not a universal placement test

Hypothesis. Under **single-replica** forceful pod-delete of `productcatalogservice`, placement will reproducibly change **reconvergence mechanisms** such as EndpointSlice state transitions, kube-proxy programming behavior, and possibly contrack disturbance, but it may **not** produce a stable ordering of user-visible outcomes or aggregate score because the direct outage of the single replica dominates the experience. That is a strong, defensible, falsifiable hypothesis. ³⁹

Independent variable. Again: `default`, `colocate`, `spread`. Keep the target fixed to `productcatalogservice` because that is what the actual `pod-delete.yaml` uses and because it is a central dependency for homepage, product pages, search, and recommendations. ⁴⁰

Dependent variables. Split them by layer.

Aggregate layer: the scenario's aggregate probe score.

Mechanism layer: EndpointSlice ready / terminating / total counts, kube-proxy network programming duration, contrack-related metrics, recovery time, scheduler/rescheduling events.

User-visible layer: route-specific latency and error rate for **dependent** routes such as homepage and product pages, plus a **control** route such as `_healthz` that should not encode the same dependency path. This layer separation is essential because your current thesis framing already argues that aggregate scores can be misleading. ⁴¹

Controls. Use the documented churn setup: 120 seconds, `CHAOS_INTERVAL=15s`, `FORCE=true`, `PODS_AFFECTED_PERC=100`, single replica, and the documented fixed load profile. Also keep cluster-level provenance fixed. The docs say that with a single replica and 100% delete, full unavailability of the target is guaranteed; that is a design feature, but also the main reason user-visible differentiation may saturate. ⁴²

Expected mechanism. Kubernetes now tracks terminating endpoints in EndpointSlices, and kube-proxy must propagate endpoint changes into dataplane state. Your new EndpointSlice snapshots are good because they provide a first-party API signature for the reconvergence story, and the kube-proxy metric definition explicitly measures propagation time from service/pod change to synced rules. You should expect a clear mechanism signature here even if aggregate score or route outcomes stay noisy. ⁴³

Metrics to collect. Use the new `endpointSlices.{preChaos,postChaos}` data, kube-proxy metrics, route-level Locust outcomes, pod readiness/recovery timing, event timelines, and any contrack reconciliation metrics you already collect. Because the pod-delete scenario currently documents **12 probes**, with score steps of roughly **8.3 points**, you should explicitly treat the aggregate score as a **coarse ordinal summary**, not a sensitive continuous measurement. ⁴⁴

Statistics. H1, H2, and H3 should be analyzed separately.

For **H1** (aggregate score instability), use variance partitioning / ICC and power/MDE reporting, exactly as your script design suggests.

For **H2** (mechanism reproducibility), use pairwise strategy comparisons with effect sizes and corrected p-values.

For **H3** (mechanism→outcome decoupling), use Spearman correlation with bootstrap CIs, a control-route

confound check, and a within-run / within-batch rank check. That is already the right structure in your script descriptions; the missing piece is the raw archive. ⁴⁵

Minimum repetitions / seeds. Same recommendation: at least **8 valid iterations per strategy**, ideally 10, preferably across more than one batch/day if you want to use the word “reproducible.” For the core three fixed strategies, seeds matter less than provenance and repeated valid iterations. For any appendix random strategy, use the documented seed list and archive it. ⁴⁶

Threats to validity. This experiment’s biggest threat is **outcome saturation**: if the target service is guaranteed to disappear, user-visible routes may all look bad regardless of placement, which weakens claims about the end-user layer. That is not fatal, but it means your conclusion must be “placement changes the reconvergence mechanism under this fault class, but not visibly enough to rank strategies at the user layer,” not “placement predicts nothing.” The scope-of-claims doc is already telling you that. ⁴⁷

Why this is academically interesting. Because it exposes a construct-validity problem. A systems thesis that shows a measurement layer is too coarse or too noisy for the underlying phenomenon is still a proper systems thesis—provided the evidence chain is clean. ⁴⁸

Network latency and loss are only worth it if you redesign them

Hypothesis. If you inject **path-scoped** network degradation on traffic to a central dependency, `spread` should suffer more than `colocate` because more calls cross nodes and therefore pay the degraded path; the effect should appear first in P95/P99 route latency and error rate. That is a strong and interesting hypothesis. But it only works if the fault is truly **path-scoped**. ⁴⁹

Independent variable. Placement strategy, and possibly fault type (`latency` vs `loss`). If you keep the current repo’s pod-level Litmus fault without redesign, the IV is undercut because the fault acts on the selected pod’s network path rather than on a sharply defined **cross-node edge** in the dependency graph. That makes placement-dependent interpretation much weaker. This is my inference from how the Litmus pod network faults are documented: they degrade the selected pod’s traffic without changing kube-proxy health status. ⁵⁰

Dependent variables. Use route-level P95/P99 latency, route failure rate, throughput, and transport-level symptoms such as retransmissions or timeout counts. Means are not the right primary outcome here; tails are. ²⁶

Controls. Same cluster, same routes, same target dependency, same load, same duration. But add one more control: verify **where** the fault actually applies with packet capture or a lightweight path sanity check. If you cannot prove that your injected latency/loss differentiates local from non-local communication, do not use it as a headline placement result. ⁵¹

Expected mechanism. If the design is right, this experiment directly tests the classic locality intuition behind scheduler and affinity work: fewer slow network hops should reduce tail latency. If the design is wrong, all traffic to the selected pod gets degraded similarly and the placement signal collapses. ⁵²

Why this is academically interesting. Potentially very interesting—but it is also the easiest experiment to get subtly wrong. Therefore it is an **ambitious extension**, not the thesis-saving core. ⁵¹

Node-cpu-hog and pod-cpu-hog should be appendix material

Hypothesis. `node-cpu-hog` may cause some throughput reduction or tail-latency inflation under dense placement, but `pod-cpu-hog` is mostly a test of container-local throttling rather than placement-sensitive shared contention in your current setup. That is exactly the distinction your updated node-memory-hog comments are already making. ⁵³

Independent variable. Placement strategy. Keep the worker shape fixed and record CPU quota/request settings. For node-cpu-hog, the scenario explicitly stresses both cores and auto-targets the node hosting `productcatalogservice`; that at least makes the experiment logically placement-aware. For pod-cpu-hog, the selected container is just throttled at its own cgroup boundary. ⁵⁴

Dependent variables. For node-cpu-hog, collect route-level P95/P99, throughput, CPU saturation, and throttling counters. For pod-cpu-hog, the important DV is almost the negative result: heavy throttling with little user-visible harm. ⁵⁵

Why these are appendix-only. Kubernetes' CPU semantics work against you here. CPU is compressible; limits are enforced by throttling; requests preserve some CPU share; and your own updated docs already say the smoke test showed 100-point scores despite increased throttling. That makes `pod-cpu-hog` ideal as a **falsification control**—a result that demonstrates why not every intuitive fault class is useful for placement analysis—but a poor candidate for the main thesis chapter. ⁵⁶

Redis latency can work, disk I/O probably will not

Redis latency is only attractive if you make placement of the `cartservice↔redis-cart` dependency pair an explicit variable. The scenario README says `cartservice` depends on `redis-cart` for session state and that the experiment injects latency between them. That can produce a real dependency-specific user-visible effect, but it is not naturally aligned with your current `productcatalog`-centered core thesis. If you keep it, make it a **separate pair-locality appendix**, not part of the core matrix. ⁵⁷

Disk I/O stress on `productcatalogservice` is the weakest serious candidate. Your own README says `productcatalog` reads product data from **embedded JSON**. That is exactly the sort of service that may be I/O-light after startup. Unless you first verify that the service is materially disk-bound during steady-state request serving, a null result here will be boring rather than illuminating. The Litmus pod-io-stress docs explain why shared disk pressure can matter in general, but your specific target service still has to actually exercise that path. ⁵⁸

Falsifiable hypotheses for the top designs

H₁ — churn / measurement-layer divergence.

Under single-replica `pod-delete` on `productcatalogservice`, `spread` and `colocate` will differ significantly in **mechanism metrics** related to reconvergence, but their **user-visible route outcomes** will not differ by a practically meaningful margin, and the aggregate probe score will not support stable strategy ranking. ⁵⁹

H₂ — contentions at the node-memory layer reappear at the user layer.

Under `node-memory-hog` targeted to the node hosting `productcatalogservice`, `colocate` will show significantly worse homepage/product-page P95/P99 latency, error rate, and blast radius than `spread`, mediated by higher counts of evictions/OOM-related restarts and more pods affected on the stressed node. ⁶⁰

H₃ — locality matters for path-specific network degradation only if the fault is actually path-specific.

A destination-scoped network latency/loss fault on traffic to a central dependency will worsen tail latency more under `spread` than under `colocate`; if the same effect is not observed under your current pod-level network fault, the correct interpretation is that the fault implementation did not isolate cross-node communication strongly enough, not that locality is irrelevant. ⁶¹

Best minimal set and stronger ambitious set

The **single best experiment set you can finish quickly** is:

Set	What to run	Why this is the best quick thesis
Recommended minimal core	<code>default</code> , <code>colocate</code> , <code>spread</code> × <code>pod-delete</code> , <code>node-memory-hog</code> , with >= 8 valid iterations per strategy per fault , archived raw data, and strict provenance gating	Gives you one churn class and one contention class, which is enough to support the fault-class / measurement-layer framing
Absolute minimum viable thesis	<code>default</code> , <code>colocate</code> , <code>spread</code> × <code>pod-delete</code> , with at least two clean batch/day runs and H1/H2/H3 analysis	Narrower, but still defensible if the title and claims explicitly say this is a single fault-class case study

The **stronger ambitious set** is:

Set	What to add	Why it makes the thesis stronger
Ambitious extension	Add a path-scoped network degradation experiment and, if time remains, a small multi-replica appendix for the pod-delete case	Path-scoped network faults test locality directly, while a multi-replica appendix addresses the obvious “but topology spread is really about replicas” objection

Literature claim to experiment mapping

The table below is the one I would actually want in the thesis, because it shows that each experiment is testing or refining an existing claim rather than just generating chaos for its own sake. The literature anchors behind the table are discussed above. ⁶²

Literature claim	Experiment that tests / builds on it	Expected result	If observed	If not observed
Spreading improves availability by separating replicas across failure domains	Single-replica <code>pod-delete</code> and <code>node-memory-hog</code> study	No direct replica-availability benefit in single-replica churn; possible collateral/ locality benefit in contention	Supports your narrower framing: placement matters through collateral effects, not classic replica HA	Means your design truly is outside the topology-spread availability story and should be framed as such
EndpointSlice / kube-proxy reconvergence is measurable during endpoint churn	<code>pod-delete</code> with EndpointSlice + kube-proxy metrics	Ready endpoints drop, possibly terminating endpoints appear, dataplane programming metrics move	Strong mechanism evidence for churn/ reconvergence claims	Suggests instrumentation or environment is not capturing the story correctly
CPU limits mostly throttle; memory pressure can evict / kill	<code>pod-cpu-hog</code> pilot vs <code>node-memory-hog</code> core	CPU hog shows throttling with weak user effect; memory hog shows stronger user-visible impact	Supports your choice of node-memory-hog as the better contention fault	If CPU hog hurts more than expected, your workload is more CPU-bound than assumed
Placement constraints can materially change latency / throughput	<code>node-memory-hog</code> and path-scoped network fault	<code>colocate</code> or <code>spread</code> should diverge depending on mechanism	Aligns ChaosProbe with placement literature under faults	Null result means either the workload is not sensitive, or your fault does not couple to placement strongly enough
Tail latency dominates interactive fan-out systems	Route-level P95/P99 on homepage / product paths	Tail metrics move before means or coarse pass/fail score	Justifies route-level tail analysis	If only means move, your load or instrumentation may be too coarse

Literature claim	Experiment that tests / builds on it	Expected result	If observed	If not observed
System-to-user degradation propagation can profile resilience	H3 mechanism→outcome analysis	Some faults will propagate cleanly; others will stop at the mechanism layer	Supports your “fault-class-specific propagation” thesis	If nothing propagates, either metrics are wrong or the setup is saturating the outcome

Claims, evidence, and defense strategy

Your new scope-of-claims page is the right instinct, but the manuscript still needs a ruthless wording audit. In particular, the words “**refuted**,” “**proven**,” “**reproducible**,” “**best strategy**,” “**resilience**,” “**causal**,” and “**generalizes**” should each trigger a conscious check. The repo’s own scope page already says you are **not** claiming a universal best strategy, not claiming a spread “refutation,” and not claiming that a single-application result generalizes broadly. The thesis prose must obey that discipline everywhere, including tables and slides. ⁶³

Claims to keep, weaken, remove, or move

Claim type	Recommendation	Better wording
“Placement effects are fault-class- and measurement-layer-specific.”	Keep	This is your core thesis statement
“Aggregate resilience scores can be misleading.”	Keep, but scope	“In this setup, the aggregate probe score was too coarse / noisy to rank strategies reliably”
“Spread is refuted” or “spread predicts nothing”	Remove	“Under this single-replica churn fault, spread did not transfer into a stable user-visible advantage”
“This is reproducible”	Weaken unless raw archives are published	“Reproduced on archived runs under the documented environment”
“Best strategy”	Remove from core thesis	“No universal ranking is justified from this study”
“Resilience” as a single scalar property	Weaken	Prefer “aggregate probe outcome score,” “mechanism metrics,” and “user-visible outcomes”
“Causal” language	Use carefully	You have manipulated placement and faults, but environment confounders still require scoped causal wording

Claim type	Recommendation	Better wording
"Generalizes to Kubernetes microservices"	Remove / move to future work	"Case-study result on Online Boutique under the documented cluster setup"

Exact evidence needed for major claims

This is the table I would use as your internal checklist before you write each claim sentence.

Claim	Required data	Required statistical test	Required figure / table	Likely examiner objection	Best defense
Aggregate score cannot rank strategies in the churn setup	Raw per-iteration score data, valid-iteration counts, provenance-clean summaries	Variance partition / ICC, MDE / power, pairwise tests with effect sizes	Score distribution plot + variance/power table	"Maybe you just under-ran it"	Show ICC / MDE and your own power analysis; if still underpowered, say so explicitly
Placement changes churn reconvergence mechanisms	EndpointSlice snapshots, kube-proxy metric, event/recovery timelines, conntrack-related metrics	Pairwise blocked comparisons + corrected p-values + effect sizes	Mechanism boxplots / violin plots + stats table	"These are noisy infra counters"	Use multiple converging mechanisms, not a single counter
Mechanism shifts do not cleanly predict user-visible outcomes under single-replica pod-delete	Same iterations: mechanism metrics plus route-level P95/P99/error-rate with control route	Spearman / bootstrap CI, within-run rank checks, control-route comparison	Mechanism-vs-outcome scatter + route/control comparison	"You picked the wrong route"	Show dependent route and explicit control route, and explain dependency graph

Claim	Required data	Required statistical test	Required figure / table	Likely examiner objection	Best defense
Node-memory-hog is more placement-sensitive than pod-cpu-hog	Clean core node-memory-hog runs plus appendix pod-cpu-hog pilot	Pairwise strategy comparisons and effect sizes for both faults	Side-by-side fault comparison table	"You changed the fault after seeing results"	Say yes, deliberately, because the earlier fault hit the wrong layer; support with Kubernetes CPU-vs-memory semantics
A universal best placement is not supported	Full core matrix, multiple measurement layers	No single test; this is a synthesis based on mixed results	Summary table across layers	"Then what did you contribute?"	You contributed a measurement result: layer-specific divergence under chaos
Results are reproducible in the documented environment	Archived summaries, manifests, hashes, batch/day IDs, strict doctor outputs, commit hashes	Repeat runs across batch/day; descriptive consistency plus inferential checks	Reproducibility / provenance table	"Where are the raw artifacts?"	Publish them, or do not say "reproducible"

Threats to validity and how to defend them

Threat class	Concrete threat	How to defend it
Internal validity	Cluster noise, background load, hidden environment drift	Batch/day IDs, same cluster config, doctor gating, artifact archiving
Construct validity	Aggregate score is too coarse; dependent routes not clearly justified	Separate aggregate / mechanism / user layers; document route dependency rationale
Statistical conclusion validity	Too many strategies, too few iterations, pseudo-replication	Narrow core matrix; block by batch/day; report effect sizes and power

Threat class	Concrete threat	How to defend it
External validity	One application, one cluster size, one CNI, one kube-proxy mode	Explicitly call it a case study; archive environment fingerprint
Instrumentation validity	kube-proxy metric depends on clock drift; connttrack behavior depends on backend mode	Record kube-proxy mode, connttrack settings, control-plane / node context
Reproducibility validity	Charts committed without raw summaries	Publish raw archives, manifests, <code>doctor --strict</code> outputs, hashes
Interpretation validity	Single-replica pod-delete saturates end-user outcomes	Say so; do not overgeneralize; contrast with node-memory-hog
Analysis validity	Fault misclassification helper can taint H1/H3	Fix the helper centrally and add tests before regenerating any plot

Hostile examiner questions and concise answer strategies

Hostile question	Concise answer strategy
"Why is this a thesis and not just a tool demo?"	"Because the thesis question is empirical: where placement effects appear under chaos, and when measurement layers diverge. The tool is the instrument, not the contribution by itself."
"Why use single replicas if topology spread is about replicated HA?"	"Because I am not claiming classic replica availability here. I use placement as an experimental variable to study collateral effects and reconvergence under faults. I state that limitation explicitly."
"Why should I believe your churn mechanism story?"	"Because it is supported by first-party Kubernetes control-plane evidence: EndpointSlice state around the fault window, kube-proxy programming metrics, and recovery/event timelines, not just one derived score."
"Why didn't you just use DeathStarBench?"	"DeathStarBench is the dominant benchmark lineage, but this thesis is a deep case study on Online Boutique. I scope the external-validity claim accordingly."
"Why drop pod-cpu-hog after investing in it?"	"Because it probes the wrong resource-control layer for this question. CPU limits are throttled; node memory pressure creates the placement-sensitive blast-radius mechanism I actually want to test."
"Are you claiming causality?"	"Only in the limited experimental sense that placement and fault were manipulated under a controlled setup. I do not claim universal causal laws for Kubernetes broadly."

Hostile question	Concise answer strategy
"If the aggregate score is noisy, doesn't that undermine your thesis?"	"No. It becomes a measurement result: this score is too coarse for ranking placement in this regime, so mechanism and route-level analyses are necessary."
"Why should I trust the scripts?"	"I archive raw runs, use strict provenance gates, and test the analysis helpers that feed the claims. I also removed a known fault-class classification bug before final analysis."
"Where is the real reproducibility package?"	"In the final thesis submission package: raw summaries, manifests, hashes, logs, doctor outputs, scripts, and exact commit IDs. Without that package I do not use the word reproducible."
"What is genuinely new relative to MicroRes?"	"MicroRes profiles degradation dissemination generally. My contribution is a placement-aware, fault-class-specific study of when that dissemination is and is not visible across measurement layers."

Thesis blueprint and completion plan

Defensible abstract

Abstract.

Modern microservice applications run on orchestration platforms whose placement decisions can affect contention, locality, and recovery behavior, but it remains unclear whether placement effects under chaos appear consistently across system mechanisms, aggregate resilience scores, and user-visible outcomes. This thesis presents **ChaosProbe**, an open-source framework for running placement-mutated chaos experiments on Kubernetes workloads using LitmusChaos and for collecting cross-layer evidence from Kubernetes, Prometheus, and workload-level probes. Using Google Online Boutique as a case study, the thesis evaluates a narrow core matrix of placement strategies (`default` , `colocate` , `spread`) under a churn fault (`pod-delete`) and a contention fault (`node-memory-hog`). The central finding is that placement effects under chaos are **fault-class-specific and measurement-layer-specific**: in a single-replica churn regime, placement can alter reconvergence mechanisms without yielding a stable user-visible ranking, while coarse aggregate probe scores can be too noisy to support defensible strategy ordering. The thesis therefore argues for separating aggregate scores, mechanism metrics, and user-visible outcomes when evaluating resilience under chaos. It also provides a reproducibility protocol based on archived run summaries, scenario hashes, batch/day identifiers, and environment provenance including kube-proxy mode and conntrack settings. ⁶⁴

Methodology and chapter blueprint

A defensible manuscript structure would be:

Opening problem. Placement literature says locality and interference matter, while chaos practice often collapses outcomes into aggregate checks. The unresolved question is whether placement effects under chaos survive across all measurement layers. ⁶⁵

Literature gap. Scheduler papers optimize placement without focusing on injected-fault observability; chaos platforms execute faults without deeply studying placement-sensitive cross-layer effects; resilience profilers connect system and user layers but do not center placement as the manipulated variable. 66

System contribution. ChaosProbe mutates placement, runs Litmus faults, collects cross-layer metrics, and now records stronger provenance: scenario hashes, batch/day IDs, kube-proxy mode, conntrack settings, EndpointSlice snapshots, and archived artifacts. 5

Experiment contribution. The thesis compares the same placement strategies under two qualitatively different fault classes—churn and contention—and analyses aggregate score, mechanism metrics, and user-visible outcomes separately. 67

Key finding. Under single-replica churn, mechanism-level changes can be real and reproducible even when user-visible outcomes do not yield a stable ranking; this makes aggregate score ranking unsafe. Under node-memory-hog, placement is expected to reappear at the user layer because the fault operates at the node-pressure layer. The first half is already the right thesis argument; the second half is the best next experiment. 68

Limitations. One application, one documented cluster environment, one main CNI / kube-proxy stack, mostly single-replica services, and noise sensitivity in some metrics. The results are case-study findings, not universal laws. 47

Practical implication. Do not rank placements from one probe aggregate. Match metrics to failure mechanism, and archive sufficient provenance to scope every claim to its environment. 69

Final figures and tables to include

Artifact	Caption	What it is supposed to prove
Figure: ChaosProbe workflow	“ChaosProbe pipeline from placement mutation to cross-layer evidence and strict provenance gating”	The thesis is a real systems artifact, not a spreadsheet
Figure: Core experiment matrix	“Core thesis matrix: three placements across churn and contention fault classes”	The design is narrow, intentional, and power-conscious
Figure: Aggregate score distributions	“Aggregate probe score by strategy under pod-delete”	H1: the coarse score does not support stable ranking
Figure: EndpointSlice / kube-proxy / conntrack mechanism plot	“Mechanism-level reconvergence signals under pod-delete by strategy”	H2: placement changes the mechanism layer
Figure: Mechanism vs route outcome scatter	“Mechanism metrics vs dependent-route tail latency, with control-route check”	H3: mechanism shifts need not transfer into user-visible differences

Artifact	Caption	What it is supposed to prove
Figure: Node-memory-hog blast radius plot	"Affected pods, evictions, and route errors under node-memory-hog by placement"	The new contention experiment is genuinely placement-sensitive
Table: Provenance fingerprint	"Environment, scenario hashes, batch/day IDs, seeds, kube-proxy mode, conntrack settings"	Claims are scoped to an archived, inspectable environment
Table: Main statistics	"Effect sizes, confidence intervals, corrected p-values, and valid iteration counts"	The analysis is statistically serious
Table: Threats to validity	"Internal, construct, statistical, and external validity threats"	You anticipated the obvious criticism
Table: Literature claim mapping	"How each experiment tests or refines existing claims"	The thesis is anchored in prior work, not random chaos

Reproducibility package requirements

The updated repo has moved in the right direction on provenance, but the final thesis package needs to go further than the current public state. The table below is the minimum package I would accept as an examiner if you want to say "reproducible in the documented environment." ⁷⁰

Category	What to archive
Raw run data	Every thesis <code>summary.json</code> ; per-iteration raw outputs; event timelines; route-level Locust CSV/JSON; Prometheus exports if they are not fully embedded
Logs and diagnostics	Litmus chaos pod logs, selected workload pod logs, <code>doctor --strict</code> outputs, any taint / exclusion reports
Environment details	Kubernetes version, CNI, container runtime, OS/kernel, kube-proxy mode, conntrack settings, worker specs, eviction thresholds
Placement provenance	Applied strategy name, placement manifest, node snapshot before chaos, node chosen by <code>TARGET_NODES=auto</code>
Scenario provenance	SHA-256 of every experiment YAML / manifest used for quoted results
Randomness control	Seeds for any randomized strategies or fault selection
Code provenance	Git commit hash for the repo; exact branch/tag used for the final figures
Analysis provenance	Exact scripts that produced each figure/table; one command block per figure

Category	What to archive
Packaging for reviewers	One compressed artifact bundle per experiment family plus a manifest index mapping figure/table numbers to raw files

Minimum viable thesis plan

If you needed the **smallest academically defensible submission**, I would do this:

Experiments.

A clean rerun of `pod-delete` only, with `default`, `colocate`, and `spread`, at least **two batch/day runs**, at least **8 valid iterations per strategy per batch**, full provenance archiving, and regenerated H1/H2/H3 analysis. ⁷¹

Claims.

Only these claims:

that the aggregate score is too coarse / noisy to rank strategies in this setup;

that mechanism-level reconvergence changes with placement under this churn fault;

that those mechanism changes do not reliably transfer to a user-visible ranking in this single-replica setting;

and that the result is scoped to this workload and environment. ⁷²

Figures.

Only five are necessary: workflow, core matrix, aggregate score distribution, mechanism plot, mechanism-vs-outcome plot. ⁷³

That is **enough** for an MSc thesis if the writing is disciplined. The title must then clearly say this is a **churn case study**, not a universal placement paper. ⁷⁴

Stronger plan if extra time remains

The **stronger but still realistic** version is:

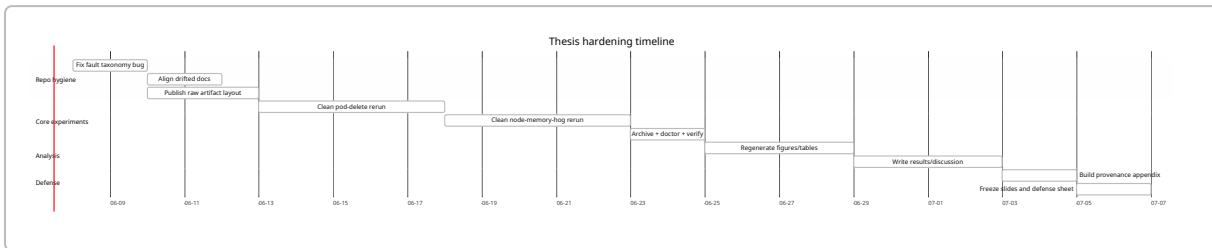
Core.

`default`, `colocate`, `spread` × `pod-delete`, `node-memory-hog`, each with at least **8–10 valid iterations per strategy**, preferably across more than one batch/day, with archived raw artifacts and strict provenance gating. ⁷⁵

Extension.

Add one **path-scoped network-loss or network-latency** experiment redesigned to isolate cross-node communication, or a **small replicated-service appendix** that addresses the “topology spread is for replicas” objection. Do **not** add all remaining faults unless you have time to archive and interpret them cleanly. Breadth is not your friend now. ⁷⁶

Completion timeline



Thesis-completion checklist

Must-have before submission

- Fix the fault-classification helper so `node-cpu-hog` cannot contaminate churn analyses. ¹⁰
- Align the scenario README with the actual core thesis scenarios and targets. ⁷⁷
- Publish raw thesis artifacts, not just charts. ⁷⁸
- Regenerate every thesis figure from archived runs with scenario hashes and strict doctor gating. ⁷⁹
- Keep the manuscript claims within the scope-of-claims page. ⁷⁴
- Include a methodology subsection that explicitly separates aggregate score, mechanism metrics, and user-visible outcomes. ⁸⁰

Should-have for a strong defense

- Complete the **node-memory-hog** core contention experiment. ⁸¹
- Add direct tests for thesis analysis scripts and any helper functions they rely on. ¹²
- Produce one provenance table mapping every final figure to commit hash, scenario hash, batch/day ID, and raw artifact filename. ⁸²
- Add a one-page appendix on why `pod-cpu-hog` was demoted. ⁸³
- If you mention Neo4j in the contributions, show at least one non-trivial thesis-relevant graph query. ⁸⁴

Nice-to-have if time remains

- Add one path-scoped network-fault extension. ⁵¹
- Add one tiny replicated-service appendix or one DeathStarBench cross-check. ⁸⁵
- Add a figure showing dependency-aware route classification and why each route is dependent or control.
- Add a script that rebuilds the entire figure set from a manifest in one command.
- Publish the raw artifacts in a long-term archive in addition to GitHub.

Unsafe claims to remove

- "ChaosProbe proves spread is not resilient."
- "We refute topology spread."
- "We found the best placement strategy."
- "The results are reproducible" without published raw archives.
- "The findings generalize to Kubernetes microservices."

- “The resilience score measures resilience” without strong construct-validity defense. ⁸⁶

One-page defense summary

If I were you, this is the defense summary I would memorize:

Problem. Placement literature shows that locality and interference can change application performance, but chaos evaluations often collapse outcomes into aggregate pass/fail summaries. That leaves an open question: when chaos is injected into a Kubernetes microservice application, does placement matter uniformly across aggregate scores, mechanism metrics, and user-visible outcomes? ⁶⁵

Approach. I built **ChaosProbe**, a placement-aware chaos experimentation framework for Kubernetes. It mutates pod placement, runs LitmusChaos faults, collects cross-layer evidence from Kubernetes, Prometheus, and load generation, and archives provenance including scenario hashes, batch/day IDs, kube-proxy mode, conntrack settings, and EndpointSlice snapshots. ⁸⁷

Core design. I intentionally narrowed the thesis to three strategies—`default`, `colocate`, `spread`—and two fault classes: `pod-delete` as churn and `node-memory-hog` as contention. This improves interpretability and statistical power. ²⁷

Main finding. In the single-replica churn setup, placement can change **reconvergence mechanisms**—the control-plane and dataplane story—but that difference does not automatically translate into a stable **user-visible ranking**, and a coarse aggregate score can therefore be misleading. That is a result about **measurement**, not just about placement. ⁸⁸

Why node-memory-hog matters. CPU hog at the container level mostly tests throttling, which is the wrong layer for the placement question here. Node memory pressure hits the eviction/OOM layer, where dense placement should create a larger blast radius. That is why node-memory-hog is the best contention experiment. ⁸⁹

Scope. I do not claim a universal best strategy, I do not claim generalization across all workloads, and I do not claim classical replica-availability results from single-replica experiments. This is a scoped case study on Online Boutique in the documented environment. ⁹⁰

Practical implication. If you want to evaluate placement under chaos, do not rank strategies from one aggregate score. Measure the mechanism layer and the user layer separately, and archive enough provenance to scope every claim to its exact environment. ⁹¹

Open questions and limitations

A few items remain unresolved from the public repo state I could inspect. I could verify the recent commit direction and several key file changes, but I could not reconstruct a literal “since your last local pull” diff because that earlier local state is not available in this session. I could also confirm that the public repo lacks

the raw thesis result archives needed to fully validate the updated H1/H2/H3 evidence chain. Those are not reasons to stop; they are the last places where the current repo is still vulnerable in a defense. 92

1 5 7 35 92 <https://github.com/yhu02/uva-msc-thesis/commits/main/>

<https://github.com/yhu02/uva-msc-thesis/commits/main/>

2 14 [uva-msc-thesis/chaosprobe/docs/explanation/hypotheses.md](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/explanation/hypotheses.md>

3 20 55 56 89 <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>

4 78 [uva-msc-thesis/chaosprobe/results/20260606-092037](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/tree/main/chaosprobe/results/20260606-092037>

6 16 17 84 87 <https://github.com/yhu02/uva-msc-thesis>

<https://github.com/yhu02/uva-msc-thesis>

8 44 73 88 [feat\(metrics\): snapshot EndpointSlice endpoint counts around fault wi...](#) · yhu02/uva-msc-thesis@ce0fd25 · GitHub

<https://github.com/yhu02/uva-msc-thesis/commit/ce0fd2545958f5fbb4dad35f8bc73c933d7b11b9>

9 25 27 30 32 34 36 42 46 47 64 67 71 75 83 [uva-msc-thesis/chaosprobe/docs/how-to/reproducing-thesis-results.md](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/how-to/reproducing-thesis-results.md>

10 [uva-msc-thesis/chaosprobe/scripts/mechanism_metrics.py](#) at main · yhu02/uva-msc-thesis · GitHub

https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/scripts/mechanism_metrics.py

11 57 58 77 [uva-msc-thesis/chaosprobe/scenarios/online-boutique](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/tree/main/chaosprobe/scenarios/online-boutique>

12 [uva-msc-thesis/chaosprobe/tests](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/tree/main/chaosprobe/tests>

13 24 <https://research.google/pubs/borg-omega-and-kubernetes/>

<https://research.google/pubs/borg-omega-and-kubernetes/>

15 63 74 86 90 [uva-msc-thesis/chaosprobe/docs/explanation/scope-of-claims.md](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/docs/explanation/scope-of-claims.md>

18 62 76 85 <https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

19 43 <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>

<https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>

21 [Principles of chaos engineering](#)

https://principlesofchaos.org/?utm_source=chatgpt.com

22 80 91 <https://arxiv.org/abs/2212.12850>

<https://arxiv.org/abs/2212.12850>

23 <https://arxiv.org/abs/2501.11107>

<https://arxiv.org/abs/2501.11107>

26 49 <https://research.google/pubs/the-tail-at-scale/>

<https://research.google/pubs/the-tail-at-scale/>

28 29 31 38 53 60 81 [uva-msc-thesis/chaosprobe/scenarios/online-boutique/node-memory-hog.yaml](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/scenarios/online-boutique/node-memory-hog.yaml>

33 37 <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>

<https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>

39 40 41 59 [uva-msc-thesis/chaosprobe/scenarios/online-boutique/pod-delete.yaml](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/scenarios/online-boutique/pod-delete.yaml>

45 48 68 72 [chore\(analysis\): reproducible H1/H3 scripts + hypotheses doc](#) · yhu02/uva-msc-thesis@8a4f65a · GitHub

<https://github.com/yhu02/uva-msc-thesis/commit/8a4f65af8ee0ee0e818da2fc5f162b108e44d92d>

50 51 61 <https://litmuschaos.github.io/litmus/experiments/categories/pods/pod-network-latency/>

<https://litmuschaos.github.io/litmus/experiments/categories/pods/pod-network-latency/>

52 65 <https://lsds.doc.ic.ac.uk/sites/default/files/medea-eurosys18.pdf>

<https://lsds.doc.ic.ac.uk/sites/default/files/medea-eurosys18.pdf>

54 [uva-msc-thesis/chaosprobe/scenarios/online-boutique/node-cpu-hog.yaml](#) at main · yhu02/uva-msc-thesis · GitHub

<https://github.com/yhu02/uva-msc-thesis/blob/main/chaosprobe/scenarios/online-boutique/node-cpu-hog.yaml>

66 https://people.eecs.berkeley.edu/~matei/papers/2013/sosp_sparrow.pdf

https://people.eecs.berkeley.edu/~matei/papers/2013/sosp_sparrow.pdf

69 <https://github.com/kubernetes/kubernetes/blob/master/pkg/proxy/metrics/metrics.go>

<https://github.com/kubernetes/kubernetes/blob/master/pkg/proxy/metrics/metrics.go>

70 79 82 [feat\(run\): persist scenario SHA-256 hashes and gate on them in doctor...](#) · yhu02/uva-msc-thesis@e0debff · GitHub

<https://github.com/yhu02/uva-msc-thesis/commit/e0debff3ad8ba76b65e0eba95f8721c048f05e9>